Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет <факультет>

Кафедра <кафедра>

Дисциплина: Тестирование программного обеспечения

ОТЧЁТ
по лабораторной работе
на тему

РАЗРАБОТКА ТЕСТ-КЕЙСОВ

Студент: <И.О. Фамилия студента>

Руководитель: <И.О. Фамилия руководителя>

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить структуру тест-кейса, освоить эквивалентное разбиение и анализ граничных значений, оформить и выполнить тест-кейсы для формы заявки на раннее подключение веб-приложения DSLKit.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Теория дана по книге С. С. Куликова «Тестирование программного обеспечения. Базовый курс» (EPAM Systems, 2015–2025), главы 2.4 и 2.7.2.

Тест-кейс — набор входных данных, условий выполнения и ожидаемых результатов, разработанный для проверки определённого свойства или поведения программы. Если не указаны входные данные, условия или ожидаемый результат, либо неясна цель, тест-кейс считается плохим. Тест-кейсы обеспечивают повторяемость проверок, передачу знаний в команде, регрессионное тестирование и улучшение требований.

Класс эквивалентности — набор данных, обрабатываемых одинаково и дающих одинаковый результат. Граничное условие — значение на границе классов эквивалентности. Для диапазона проверяют последнее допустимое и первое недопустимое значения, а также «опасные» значения вроде нуля и пустой строки. Позитивные тест-кейсы берут из допустимых классов, негативные из недопустимых.

3 ХОД РАБОТЫ

3.1 Выбор use case и требования

Тестируется вариант использования UC-01, описанный ниже. Проверку полей выполняет функция validateWaitlist (файл site/js/utils/validation.js), которая возвращает список неверных полей. Проект развёрнут локально командой npm start.

Название.

– **UC-01. Подать заявку в лист ожидания (форма «Записаться в бету»** на главной странице DSLKit).

Идентификатор, версия, приоритет.

– UC-01, версия 1.0, приоритет High: единственный способ оставить контакты для получения приглашения в бету.

Система и модуль.

– **Система:* DSLKit — *веб-приложение «Редактор диаграмм как кода: слева текст, справа диаграмма», статический сайт без серверной части (каталог* site/*);

– **Модуль:** «Лист ожидания беты» — форма заявки на главной странице (раздел «Записаться в бету»);

– **Ссылка на модуль: http://localhost:5173/#waitlist (запуск:* npm start*);

– **Ссылка на исходный код: https://github.com/KsushaKhadzhinova/DSLKit-Project, файлы* site/index.html *(форма* #waitlist-form*) и* site/js/components/waitlist-form.js*.

Краткое описание цели и задачи.

– Посетитель оставляет имя, почту, нотацию диаграмм и размер команды, чтобы команда DSLKit прислала ему приглашение, когда откроется бета. Задача формы: принять только корректную заявку, сохранить её и показать страницу благодарности.

Основной актор.

– Посетитель сайта (неавторизованный). Внешних систем и сервера в сценарии нет: заявка остаётся в браузере посетителя.

Как перейти к форме.

Форма находится на главной странице сайта в разделе «Записаться в бету» (якорь* #waitlist*), доступна всем посетителям без регистрации и авторизации. Перейти к ней можно так:

– **1. Шапка главной страницы, кнопка «В бету» (en: «Join the beta»),* site/index.html*, строка 67;

– **2. Первый экран, кнопка «Записаться в бету»**, строка 88;

– **3. Раздел «Нотации»: шесть ссылок «Записаться в бету: UML / BPMN / ERD / DFD / IDEF0 / сети Петри»**, строки 127–172;

– **4. Страницы* privacy.html *и* thanks.html*: кнопка «В бету» в шапке ведёт на* ./#waitlist*;

– *5.* Прямой адрес: http://localhost:5173/#waitlist (русская версия), http://localhost:5173/en/#waitlist (английская);

– *6.* Клавиатура: Tab до кнопки или ссылки и Enter; ссылка-якорь прокручивает страницу к форме;

– *7. React-приложение (*react-app*): кнопка «Записаться в бету»** в шапке (UC-04, только сообщение, заявка не сохраняется).

Предусловия.

– 1. Открыта главная страница: русская (/) или английская (/en/);

– 2. Раздел **«Записаться в бету» показан, форма пуста: список «Нотация» стоит на пункте «Выберите нотацию», в поле «Размер команды»** значение 0, флажок согласия снят;

– 3. Скрипт *site/js/script.js* загружен (иначе форма не работает).

Основной поток событий.

– 1. Посетитель переходит к форме одним из способов раздела «Как перейти к форме», например нажимает **«В бету» (или «Записаться в бету»); страница прокручивается к разделу* #waitlist*;

– 2. Посетитель вводит имя в поле **«Имя\»** (подсказка «От 2 до 60 букв. Допустимы пробел, дефис и апостроф.»**);

– 3. Посетитель вводит адрес в поле **«Электронная почта\»** (подсказка «Адрес вида name@example.com. На него придёт приглашение.»**);

– 4. Посетитель выбирает нотацию в списке «Нотация\»: *IDEF0, DFD, ERD, BPMN, UML или «Сети Петри»**;

– 5. Посетитель вводит целое число в поле **«Размер команды\»** (подсказка «Целое число от 0 до 500. Укажите 0, если работаете один.»**);

– 6. Посетитель при желании вводит текст в поле **«Комментарий (по желанию)»**;

– 7. Посетитель отмечает флажок **«Согласен получать письма о запуске беты и с политикой конфиденциальности\»****;

– 8. Посетитель нажимает кнопку **«Отправить»**;

– 9. Система считывает шесть полей и проверяет каждое по правилам П1–П6 из таблицы «Структура данных» (функция *validateWaitlist*);

– 10. Система формирует запись {name, email, notation, team, comment, createdAt}, где name, email и comment очищены от пробелов по краям, team приведён к числу, createdAt — время в формате ISO 8601; добавляет запись в массив *LocalStorage* по ключу *dslkit.waitlist* и пишет в консоль **«DSLKit: заявка сохранена в LocalStorage»** с названием нотации;

– 11. Система открывает страницу благодарности *thanks.html* (для английской версии en/*thanks.html*) с заголовком **«Спасибо, вы в списке беты»**;

– * — обязательное поле. В интерфейсе сайта звёздочек нет, обязательность указана фразой «Все поля, кроме комментария, обязательны».

Альтернативные потоки.

– **A1. Одно или несколько полей не прошли проверку (шаг 9): под каждым неверным полем показывается красное сообщение (role=alert, например*

*«Имя не принято: поле пустое, короче двух символов или содержит цифры и знаки…»), поле получает aria-invalid=true, фокус переходит на первое неверное поле в порядке имя, почта, нотация, команда, комментарий, согласие. Заявка не сохраняется**, страница не меняется. Поле с ошибкой перепроверяется при каждом вводе, остальные при выходе из поля. После исправления сообщение скрывается. Сценарий продолжается с шага 8;

– **A2. Согласие не отмечено (шаг 9): применяется A1; сообщение «Без согласия мы не можем отправить приглашение. Поставьте галочку, чтобы продолжить.»**;

– **A3**. Посетитель закрывает страницу или уходит со страницы до шага 8: данные не сохраняются, состояние системы не меняется;

– **A4. Повторная заявка с тем же адресом (шаг 10): принимается как новая отдельная запись**, сообщения о дубликате нет (дефект D-1);

– **A5. Повторное нажатие «Отправить»** после успешной отправки (шаг 11): игнорируется, вторая запись не создаётся (флаг submitting).

Постусловия.

– Успех: массив *dslkit.waitlist* в *LocalStorage* длиннее на одну запись, открыта страница *thanks.html* (en/*thanks.html*);

– Ошибка проверки (A1, A2): записей не добавилось, посетитель остаётся на главной странице, введённые значения остаются в полях.

Исключительные ситуации.

– **E1. Запись в* LocalStorage *не удалась (хранилище переполнено или запрещено, шаг 10): метод* storage.set *возвращает false, но система всё равно открывает страницу благодарности; сообщения об ошибке нет, заявка потеряна** (дефект D-2);

– **E2.* LocalStorage *недоступен совсем (*getStorage* использует временное хранилище в памяти): заявка живёт только до закрытия страницы, посетитель видит страницу благодарности.

Связанные требования.

– **R4** — Форма заявки и валидация;

– **R1 — Доступность страниц (главная,* thanks.html*,* privacy.html*);

– **R6** — Разметка и локализация (русская и английская версии);

– **R9** — Локальное хранилище.

Подзадачи.

– **UC-01.1** — Показ формы и подсказок к полям;

– **UC-01.2** — Чтение, очистка и проверка значений по П1–П6;

– **UC-01.3** — Показ ошибок, фокус и повторная проверка;

– **UC-01.4 — Сохранение заявки в* LocalStorage*;

– **UC-01.5** — Переход на страницу благодарности.

Связанные задачи и use case.

– **UC-00 — Прочитать политику конфиденциальности (ссылка в тексте согласия,* privacy.html*);

– **UC-02 — Выбрать нотацию на карточке и перейти к записи (ссылки*

*«Записаться в бету: UML», «…: BPMN»**);

– **UC-03 — Авторизация и вход в приложение: в текущей версии* не реализована*, форма и приложение (*site/app/*) работают без входа; задача на будущее, после появления серверной части;

– **UC-04 — Запись в бету из React-приложения (*react-app/src/App.jsx*, лаб. 7);

– **D-1** — Повторные заявки принимаются;

– **D-2** — Потеря заявки при сбое хранилища.

Реализация.

– **IMPL-01* — site/index.html*: разметка формы* #waitlist-form*;

– **IMPL-02* — site/js/components/waitlist-form.js*:* initWaitlistForm*, показ ошибок, отправка;

– **IMPL-03* — site/js/utils/validation.js*:* validateWaitlist* и правила П1–П6;

– **IMPL-04* — site/js/utils/storage.js*: createStorage,* getStorage*;

– **IMPL-05* — site/thanks.html *и* site/en/thanks.html*: страница благодарности.

Тест-кейсы и проверки.

– **TC-WL-001** — Все поля корректны;

– **TC-WL-002…007** — Имя: границы и недопустимые символы (П1);

– **TC-WL-008…011** — Почта: формат и длина (П2);

– **TC-WL-012…017** — Размер команды: границы и тип (П4);

– **TC-WL-018** — Нотация вне списка (П3);

– **TC-WL-019, 020** — Комментарий: граница 500 (П5);

– **TC-WL-021** — Согласие не дано (П6);

– **TC-WL-022** — Пустая форма (А1);

– **SM-07** — Форма заявки есть на главной;

– **CP-35** — Валидация корректной заявки;

– **CP-36** — У полей есть подписи и сообщения об ошибках.

Скриншоты интерфейса.

– **SCR-01** — Главная страница, русская версия (01-home.png);

– **SCR-02** — Главная страница, английская версия (02-home-en.png);

– **SCR-03** — Форма заявки: ошибки при пустой отправке (03-form-errors.png);

– **SCR-04** — Форма заявки: заполнена корректными данными (04-form-filled.png);

– **SCR-05** — Страница благодарности (05-form-submitted.png);

– **SCR-06 — Запись заявки в* LocalStorage* (12-uc-storage-record.png);

– **SCR-07** — Сбой записи: страница благодарности без ошибки (13-uc-storage-failure.png).

Ссылки.

Страницы проекта (при запущенном npm start):

– http://localhost:5173/ — главная страница (русская версия);

– http://localhost:5173/#waitlist — раздел «Записаться в бету»;
– http://localhost:5173/thanks.html — страница благодарности;
– http://localhost:5173/privacy.html — политика конфиденциальности;
– http://localhost:5173/en/ — главная страница (английская версия);
– http://localhost:5173/en/thanks.html — страница благодарности (английская версия).

Репозиторий и источники:

– https://github.com/KsushaKhadzhinova/DSLKit-Project — репозиторий проекта;
– https://svyatoslav.biz/software_testing_book/ — книга «Тестирование программного обеспечения. Базовый курс»;
– https://github.com/KsushaKhadzhinova/report-gen — генератор отчётов.

Таблица 3.1 – Структура данных заявки (сущность WaitlistEntry)

Поле	Тип, длина	Обязат., уникальность	Маска	По умолч.	Связь	Прав.
name*	string, 2–60	Да; не уникально	буквы, пробел, апостроф, дефис; первая буква	—	Связей нет	П1
email*	string, 6–120	Да; не уникально	name@domain.zone	—	Адрес получателя приглашения	П2
notation*	enum, 3–5	Да; не уникально	один из шести: idef0 … petri	не выбрано	FK → Notation.id (N:1), справочник NOTATIONS в validation.js	П3
team*	integer, 0–500 (до 3 цифр)	Да; не уникально	целые цифры 0–500	0	Связей нет	П4
comment	string, 0–500	Нет; не уникально	без маски	пусто	Связей нет	П5

Поле	Тип, длина	Обязат., уникаль-ность	Маска	По умолч.	Связь	Прав.
consent*	boolean, —	Да, значение true; —	только true	false	→ документ «Полити-ка конфиден-циально-сти» (privacy.html)	П6
createdAt	string, 24	Да; не уникально	ISO 8601, UTC	текущее время	Связей нет	—

Хранилище: массив JSON в LocalStorage, ключ dslkit.waitlist. Первичного ключа нет. Связь «заявка → нотация» N:1 со справочником из шести значений. Звёздочка у имени поля означает обязательное поле.

Таблица 3.2 – Данные формы: типы и допустимые значения

Поле	Тип	Обяз.	По умолч.	Допустимо	Недопусти
name* (#name)	string; input type=text	Да	пусто	От 2 до 60 символов; первая буква, далее буквы любого алфавита, пробел, апостроф, дефис; пробелы по краям игно-рируются. Примеры: «Анна Иванова», «Ян», «O’Neil-Smith»	Пусто; 1 символ; 6 и более; цифры («Anna1») знаки («Анна!») первый символ не буква («-Анна»)

Поле	Тип	Обяз.	По умолч.	Допустимо	Недопусти
email* (#email)	string; input type=email	Да	пусто	Вид name@domain.zone, зона не короче 2 символов, до 120 символов. Пример: anna@example.com	Пусто; без @; без зон («anna@ex пробелы внутри; 12 символ и более
notation* (#notation)	enum; select, значение string	Да	не выбрано	Ровно одно из: idef0, dfd, erd, bpmn, uml, petri	Пусто (пункт «Выберит нотацию» любое другое значение («xyz»); заглавные буквы («IDEF0»)
team* (#team)	integer; input type=number (в коде допускается и строка из цифр)	Да	0	Целое от 0 до 500 включи-тельно. Примеры: 0, 5, 500, «12»	Меньше 0 («-1»); больше 50 («501»); дробное («2.5»); не число («abc»); пусто
comment (#comment)	string; textarea	Нет	пусто	От 0 до 500 символов (пусто допустимо)	501 симво и более (в браузере ввод ограничен maxlength
consent* (#consent)	boolean; checkbox	Да	false (снят)	true (флажок отмечен)	false (флажок снят)

Поле	Тип	Обяз.	По умолч.	Допустимо	Недопуст
createdAt (формирует система)	string; время ISO 8601 (UTC)	Нет (не вводится)	текущее время в момент отправки	Строка вида 2026-10-25T12:00:00.000Z	Не вводит посетите- лем

3.2 Диаграммы процесса

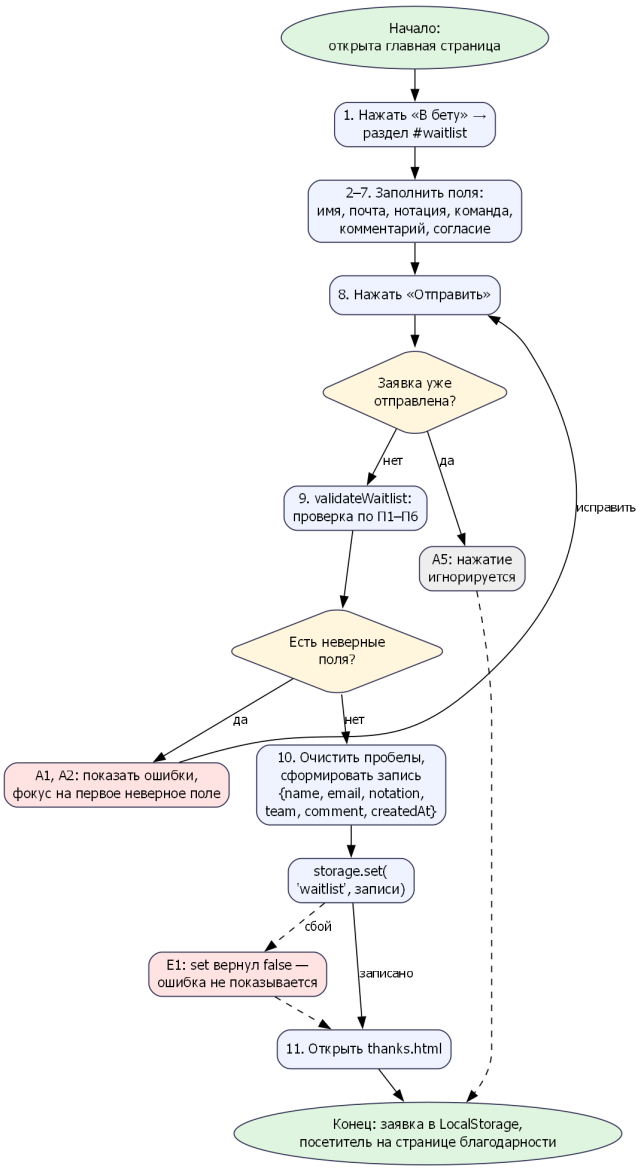


Рисунок 3.1 – Блок-схема сценария UC-01

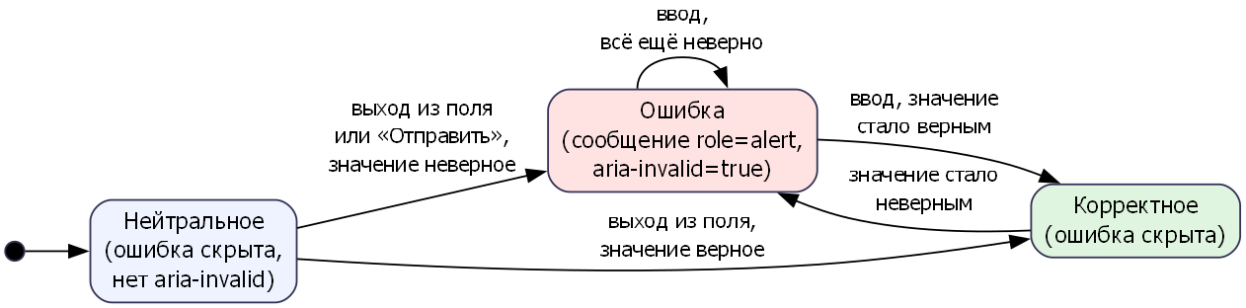


Рисунок 3.2 – Состояния поля формы при проверке

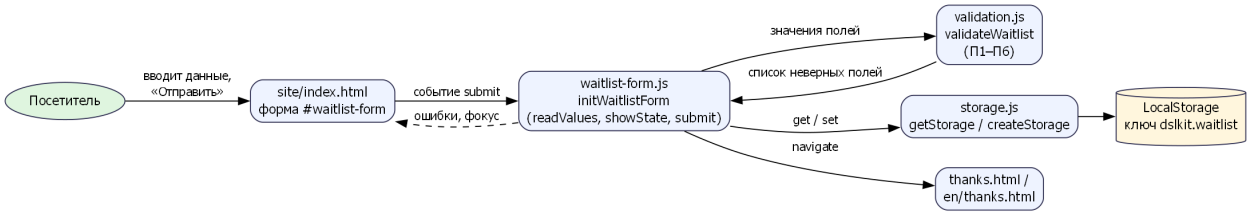


Рисунок 3.3 – Компоненты и обмен данными

Трассировка: правило П1 проверяют TC-WL-002…007, П2 008…011, П3 018, П4 012…017, П5 019 и 020, П6 021, поток А1 022.

Требования к полям приведены в таблице ниже.

Таблица 3.3 – Требования к полям формы

Поле	Правило
name	Начинается с буквы; буквы, пробел, апостроф, дефис; длина от 2 до 60
email	Формат адреса с доменом верхнего уровня; длина не более 120
notation	Одно из: idef0, dfd, erd, bpmn, uml, petri
team	Целое число от 0 до 500 (допустима строка из цифр)
comment	Не более 500 символов, необязательное
consent	Только true

3.3 Классы эквивалентности и границы

Для каждого поля выделены допустимый и недопустимые классы; границы проверяются парами «последнее допустимое и первое недопустимое» значение (таблица ниже).

Таблица 3.4 – Классы эквивалентности и граничные значения

Поле	Допустимые	Недопустимые	Границы
name	Буквы 2–60	Цифры, 1 символ, более 60	1 / 2 / 60 / 61
email	Корректный адрес	Без @, без домена, более 120	120 / 121
team	0–500	Отрицательное, больше 500, не число	-1 / 0 / 500 / 501
notation	Из списка	Любое другое значение	—
comment	0–500	Более 500	500 / 501
consent	true	false, отсутствует	—

3.4 Тест-кейсы и результаты

Разработано 22 тест-кейсов: позитивные, негативные и граничные. Идентификаторы в таблице сокращены: полный вид TC-WL-NNN. В столбце «Данные» указаны только поля, отличающиеся от базового корректного набора (name=Анна Иванова, email=anna@example.com, notation=idef0, team=5, comment пустой, consent=true). Общие предусловия: проект запущен, форма открыта, поля пусты. Постусловие: данные не сохраняются, состояние системы не меняется. Шаги общие: 1) подать набор данных на вход проверки формы; 2) получить список неверных полей. Ожидаемый результат указан в таблице.

Таблица 3.5 – Тест-кейсы и результаты выполнения

ID	Тип, приоритет	Заголовок	Данные	Ожидаемый	Фактичес
WL-001	позитивный, High	Все поля корректны	все поля корректны	ошибок нет	ошибок не (Passed)
WL-002	граничный, High	Имя из 1 символа (ниже границы 2)	name=”A”	name	name (Passed)
WL-003	граничный, Medium	Имя из 2 символов (нижняя граница)	name=”Ян”	ошибок нет	ошибок не (Passed)
WL-004	граничный, Medium	Имя из 60 символов (верхняя граница)	name=aaaaa… (60)	ошибок нет	ошибок не (Passed)

ID	Тип, приоритет	Заголовок	Данные	Ожидаемый	Фактичес
WL-005	граничный, Medium	Имя из 61 символа (выше границы)	name=aaaaa… (61)	name	name (Passed)
WL-006	негативный, High	Имя с цифрой	name=”Anna1”	name	name (Passed)
WL-007	позитивный, Low	Имя с апострофом и дефисом	name=”O’Neil-Smith”	ошибок нет	ошибок не (Passed)
WL-008	негативный, High	Email без символа @	email=anna.… (16)	email	email (Passed)
WL-009	негативный, Medium	Email без домена верхнего уровня	email=”anna@example”	email	email (Passed)
WL-010	граничный, Medium	Email длиной 120 символов	email=aaaaa… (120)	ошибок нет	ошибок не (Passed)
WL-011	граничный, Medium	Email длиной 121 символ	email=aaaaa… (121)	email	email (Passed)
WL-012	граничный, High	Размер команды 0 (нижняя граница)	team=0	ошибок нет	ошибок не (Passed)
WL-013	граничный, High	Размер команды 500 (верхняя граница)	team=500	ошибок нет	ошибок не (Passed)
WL-014	граничный, High	Размер команды 501 (выше границы)	team=501	team	team (Passed)
WL-015	граничный, Medium	Размер команды -1 (ниже границы)	team=-1	team	team (Passed)

ID	Тип, приоритет	Заголовок	Данные	Ожидаемый	Фактичес
WL-016	негативный, Medium	Размер команды — не число	team=”abc”	team	team (Passed)
WL-017	позитивный, Low	Размер команды строкой ”12”	team=”12”	ошибок нет	ошибок не (Passed)
WL-018	негативный, High	Нотация вне списка	notation=”xyz”	notation	notation (Passed)
WL-019	граничный, Low	Комментарий 500 символов	comment=xxxxx (500)	ошибок нет	ошибок не (Passed)
WL-020	граничный, Low	Комментарий 501 символ	comment=xxxxx (501)	comment	comment (Passed)
WL-021	негативный, High	Согласие на обработку не дано	consent=false	consent	consent (Passed)
WL-022	негативный, High	Пустая форма	name=undefined; email=undefined; notation=undefined; team=undefined; comment=undefined; consent=undefined	name, email, notation, team, consent	name, ema notation, team, consent (Passed)

4 АНАЛИЗ РЕЗУЛЬТАТОВ

Выполнено 22 тест-кейсов, пройдено 22, не пройдено 0. Дефектов не выявлено: проверка полей соответствует требованиям на всех границах. Попутно исправлена ошибка самих тестов на этапе отладки (неверный формат тестовых данных), к продукту она отношения не имеет.

5 ВЫВОД

Освоены оформление тест-кейсов по шаблону, эквивалентное разбиение и анализ граничных значений. Тест-кейсы автоматизированы (команда npm run tc:lab2) и повторяются без участия человека.